

KUBERNETES NETWORKING & SECURITY – INTERVIEW Q/A



Shruthi Chikkela
Learnwithshruthi

1. "What happens when a pod on Node A talks to a pod on Node B?"

Learning Mode

- Kubernetes uses a **flat pod network** every pod gets its own IP, routable across the cluster.
- When Pod-A (on Node A) sends traffic to Pod-B (on Node B):
 1. Pod-A sends traffic to Pod-B's IP.
 2. The container's virtual Ethernet pair sends it into Node A's network namespace.
 3. The **CNI plugin** (Flannel, Calico, Cilium, etc.) checks the destination IP, sees it belongs to Node B's subnet.
 4. Traffic is encapsulated (VXLAN, IP-in-IP, or eBPF) and sent over the underlying network to Node B's node IP.
 5. Node B's CNI decapsulates and routes it into Pod-B's veth interface.
- No NAT between pods — that's why services like NetworkPolicies work by IP.

Interview Mode

"Pods in Kubernetes communicate over a flat IP network. When Pod-A on Node A talks to Pod-B on Node B, the CNI plugin encapsulates traffic and routes it over the underlying node network. Node B's plugin decapsulates and delivers it to Pod-B's virtual interface. This is transparent to the pods, they just see direct IP-to-IP communication."

2. "How does kube-proxy route LoadBalancer traffic internally?"

Learning Mode

- In cloud providers (like Azure), a LoadBalancer service gets an external IP.
- kube-proxy maintains **iptables/ipvs rules** mapping that external IP:port to the ClusterIP of the service.
- ClusterIP then load-balances across endpoints (Pods).
- Path: **External IP → NodePort → ClusterIP → Pod**.
- Cloud LB sends traffic to Node IP on NodePort → kube-proxy DNATs → Pod IP.

Interview Mode

"For a LoadBalancer service, kube-proxy programs iptables or IPVS rules so that external LB traffic lands on a NodePort, then DNATs to the ClusterIP and finally to the Pod IP. The cloud LB just delivers packets to the node — kube-proxy handles the rest internally."

3. "Why would a NetworkPolicy allow traffic but it still fails?"

Learning Mode

Possible reasons:

- The CNI plugin doesn't support NetworkPolicies (e.g., Flannel without policy controller).
- Policies allow traffic at Kubernetes level but **node-level firewalls** (Azure NSGs, iptables) still block it.
- Application is listening on a different port or interface.
- DNS resolution failing (policy allows IP but not DNS).

Interview Mode

"NetworkPolicy success depends on the CNI. If the plugin doesn't enforce it, or if lower-level firewalls block the path, traffic fails. It can also fail if the app isn't listening correctly or if supporting traffic like DNS isn't allowed."

4. "What breaks if CoreDNS dies?"

Learning Mode

- All in-cluster DNS resolution stops:
 - Service name lookups (my-service.default.svc.cluster.local) fail.
 - External domain lookups fail unless pods bypass CoreDNS.
- Direct Pod IP communication still works, but anything using service discovery breaks.

Interview Mode

"Without CoreDNS, Kubernetes loses service discovery. Pods can still talk via IP, but any DNS-based connection to services or external hosts fails."

5. "Explain how Cilium differs from Flannel and when you'd use each."

Learning Mode

- **Flannel:** Simple overlay (VXLAN/IP-in-IP), focus is on connectivity, not policy. Lightweight, no advanced security.
- **Cilium:** eBPF-based, faster, supports Layer 7 policies, visibility, and service mesh-like features.
- Use Flannel for simple, low-overhead setups.
- Use Cilium for security, observability, and high-performance networking.

Interview Mode

"Flannel provides basic pod networking using overlays. Cilium uses eBPF for high-performance networking, advanced security, and observability. Choose Flannel for simple clusters, Cilium for complex, security-sensitive, or performance-critical workloads."

6. "Can a pod connect to itself via Service IP? Why or why not?"

Learning Mode

- Yes, if hairpinMode is enabled in kubelet.
- kube-proxy DNATs traffic from Service IP to pod IP, even if it's the same pod.
- Without hairpinMode, packets are dropped because Linux sees them looping back on the same interface incorrectly.

Interview Mode

"A pod can connect to itself via its Service IP if hairpinMode is enabled. kube-proxy DNATs to its own IP. Without hairpinMode, the kernel drops the traffic."

7. "How do you simulate a broken path between two pods?"

Learning Mode

- Easiest: use `kubectl exec` into one pod and run:
`iptables -A OUTPUT -d <target-pod-ip> -j DROP`
- Or apply a restrictive NetworkPolicy denying that pod's traffic.
- Or simulate at node level with Azure NSG rules blocking pod CIDRs.

Interview Mode

"You can simulate a broken path by blocking the target pod's IP with iptables inside the source pod, or by applying a deny NetworkPolicy. On Azure, you could also block the pod CIDR at the NSG level."

8. "What does hostNetwork: true really do and why's it risky?"

Learning Mode

- Pod shares node's network namespace:
 - Same IP as node.
 - Can bind to host ports directly.
- Risks:
 - Pod can sniff all node traffic.
 - Port conflicts with node processes.
 - Easier lateral movement if compromised.

Interview Mode

"hostNetwork: true puts a pod in the node's network namespace, sharing its IP and ports. It's risky because it can sniff host traffic, cause port conflicts, and expand the blast radius if compromised."

If you want, I can also give you a **single-page "memory map" diagram** so you can recall these instantly in interviews without re-reading long notes. That would make this stick. Do you want me to prepare that?

FOR ANY REALTIME HANDSON PROJECTS

 Follow



Shruthi Chikkela
Learnwithshruthi